

Write code that uses standard J2SE APIs in the `java.util` and `java.util.regex` packages to format or parse strings or streams. For strings, write code that uses the `Pattern` and `Matcher` classes and the `String.split(...)` method. Recognize and use regular expression patterns for matching (limited to: `.` (dot), `*` (star), `+` (plus), `?`, `\d`, `\s`, `\w`, `[]`, `()`). The use of `*`, `+`, and `?` will be limited to greedy quantifiers, and the parenthesis operator will only be used as a grouping mechanism, not for capturing content during matching. For streams, write code using the `Formatter` and `Scanner` classes and the `PrintWriter.format/print` methods. Recognize and use formatting parameters (limited to: `%b`, `%c`, `%d`, `%f`, `%s`) in format strings.

[Prev](#)

Chapter 3. API Contents

[Next](#)

Write code that uses standard J2SE APIs in the `java.util` and `java.util.regex` packages to format or parse strings or streams. For strings, write code that uses the `Pattern` and `Matcher` classes and the `String.split(...)` method. Recognize and use regular expression patterns for matching (limited to: `.` (dot), `*` (star), `+` (plus), `?`, `\d`, `\s`, `\w`, `[]`, `()`). The use of `*`, `+`, and `?` will be limited to greedy quantifiers, and the parenthesis operator will only be used as a grouping mechanism, not for capturing content during matching. For streams, write code using the `Formatter` and `Scanner` classes and the `PrintWriter.format/print` methods. Recognize and use formatting parameters (limited to: `%b`, `%c`, `%d`, `%f`, `%s`) in format strings.

The Java 2 Platform, Standard Edition (J2SE), version 1.4, contains a new package called `java.util.regex`, enabling the use of regular expressions. Now functionality includes the use of meta characters, which gives regular expressions versatility.

A regular expression, specified as a string, must first be compiled into an instance of this class. The resulting pattern can then be used to create a `Matcher` object that can match arbitrary character sequences against the regular expression. All of the state involved in performing a match resides in the matcher, so many matchers can share the same pattern.

A typical invocation sequence is thus:

```
Pattern p = Pattern.compile("a*b");
Matcher m = p.matcher("aaaaab");
boolean b = m.matches();
```

A `matches` method is defined by `Pattern` class as a convenience for when a regular expression is used just once. This method compiles an expression and matches an input sequence against it in a single invocation. The statement:

```
boolean b = Pattern.matches("a*b", "aaaaab");
```

is equivalent to the three statements above, though for repeated matches it is less efficient since it does not allow the compiled pattern to be reused.

Instances of `Pattern` class are immutable and are safe for use by multiple concurrent threads. Instances of the `Matcher` class are not safe for such use.

Character classes

<code>[abc]</code>	a, b, or c (simple class)
<code>[^abc]</code>	Any character except a, b, or c (negation)
<code>[a-zA-Z]</code>	a through z or A through Z, inclusive (range)
<code>[a-d[m-p]]</code>	a through d, or m through p: <code>[a-dm-p]</code> (union)
<code>[a-z&&[def]]</code>	d, e, or f (intersection)
<code>[a-z&&[^bc]]</code>	a through z, except for b and c: <code>[ad-z]</code> (subtraction)
<code>[a-z&&[^m-p]]</code>	a through z, and not m through p: <code>[a-lq-z]</code> (subtraction)

Predefined character classes

.	Any character (may or may not match line terminators)
\d	A digit: [0-9]
\D	A non-digit: [^0-9]
\s	A whitespace character: [\t\n\x0B\f\r]
\S	A non-whitespace character: [^\s]
\w	A word character: [a-zA-Z_0-9]
\W	A non-word character: [^\w]

Pattern Class

An instance of the Pattern class represents a regular expression that is specified in string form in a syntax similar to that used by Perl.

A regular expression, specified as a string, must first be compiled into an instance of the Pattern class. The resulting pattern is used to create a Matcher object that matches arbitrary character sequences against the regular expression. Many matchers can share the same pattern because it is stateless.

The compile method compiles the given regular expression into a pattern, then the matcher method creates a matcher that will match the given input against this pattern. The pattern method returns the regular expression from which this pattern was compiled.

The split method is a convenience method that splits the given input sequence around matches of this pattern. The following example uses split to break up a string of input separated by commas and/or whitespace:

```
import java.util.regex.*;

public class Splitter {
    public static void main(String[] args) throws Exception {
        // Create a pattern to match breaks
        Pattern p = Pattern.compile("[,\\s]+");
        // Split input with the pattern
        String[] result = p.split("one,two, three  four ,  five");
        for (int i=0; i<result.length; i++) {
            System.out.println("|" + result[i] + "|");
        }
    }
}
```

The output:

```
|one|
|two|
|three|
|four|
|five|
```

Matcher Class

Instances of the Matcher class are used to match character sequences against a given string sequence pattern. Input is provided to matchers using the CharSequence interface to support matching against characters from a wide variety of input sources.

A matcher is created from a pattern by invoking the pattern's matcher method. Once created, a matcher can be used to perform three different kinds of match operations:

- The matches method attempts to match the entire input sequence against the pattern.
- The lookingAt method attempts to match the input sequence, starting at the beginning, against the pattern.
- The find method scans the input sequence looking for the next sequence that matches the pattern.

Each of these methods returns a boolean indicating success or failure. More information about a successful match can be obtained by querying the state of the matcher.

The Matcher class also defines methods for replacing matched sequences by new strings whose contents can, if desired, be computed from the match result.

The appendReplacement method appends everything up to the next match and the replacement for that match. The appendTail appends the strings at the end, after the last match.

The following code samples demonstrate the use of the java.util.regex package. This code writes "One dog, two dogs in the yard" to the standard-output stream:

```
import java.util.regex.*;

public class Replacement {
    public static void main(String[] args) throws Exception {
        // Create a pattern to match cat
        Pattern p = Pattern.compile("cat");
        // Create a matcher with an input string
        Matcher m = p.matcher("One cat, two cats in the yard");
        StringBuffer sb = new StringBuffer();
        boolean result = m.find();
        // Loop through and create a new String with the replacements
        while(result) {
            m.appendReplacement(sb, "dog");
            result = m.find();
        }
        // Add the last segment of input to the new String
        m.appendTail(sb);
        System.out.println(sb.toString());
    }
}
```

Quantifiers

Quantifiers specify the number of occurrences of a pattern. This allows us to control how many times a pattern occurs in a string. Table summarizes how to use quantifiers:

Table 3.1. Quantifiers

Greedy Quantifiers	Reluctant Quantifiers	Possessive Quantifiers	Occurrence of a pattern X
X?	X??	X?+	X, once or not at all
X*	X*?	X*+	X, zero or more times
X+	X+?	X++	X, one or more times
X{n}	X{n}?	X{n}+	X, exactly n times
X{n,}	X{n,}?	X{n,}+	X, at least n times
X{n,m}	X{n,m}?	X{n,m}+	X, at least n but not more than m times

The first three columns show regular expressions that represent a set of strings in which X loops occur. The last column describes the meaning of its corresponding regular expressions. There are three types of quantifiers to specify each kind of

pattern occurrence. These three types of quantifiers are different in usage. It's important to understand the meaning of the metacharacters used in quantifiers before we explain the differences.

The most general quantifier is {n,m}, where n and m are integers. X{n,m} means a set of strings in which X loops at least n times but no more than m times. For instance, X{3, 5} includes XXX, XXXX, and XXXXX but excludes X, XX, and XXXXXX.

Even though we have the above metacharacters to control occurrence, there are several other ways to match a string with a regular expression. This is why there is a greedy quantifier, reluctant quantifier, and possessive quantifier in each case of occurrence.

A greedy quantifier forces a Matcher to digest the whole inputted string first. If the matching fails, it then forces the Matcher to back off the inputted string by one character, check matching, and repeat the process until there are no more characters left.

A reluctant quantifier, on the other hand, asks a Matcher to digest the first character of the whole inputted string first. If the matching fails, it appends its successive character and checks again. It repeats the process until the Matcher digests the whole inputted string.

A possessive quantifier, unlike the other two, makes a Matcher digest the whole string and then stop.

Table below helps to understand the difference between the greedy quantifier (the first test), the reluctant quantifier (the second test), and the possessive quantifier (the third test). The string content is "whelloworldwwwwhelloworldwww"

Table 3.2. Difference between quantifiers

Regular Expression	Result
. *hello	Found the text "whelloworldwwwhello" starting at index 0 and ending at index 17.
. *?hello	Found the text "hello" starting at index 0 and ending at index 6. Found the text "wwwwwwhello" starting at index 6 and ending at index 17.
. *+hello	No match found.

Capturing groups

The above operations also work on groups of characters by using capturing groups. A capturing group is a way to treat a group of characters as a single unit. For instance, (java) is a capturing group, where java is a unit of characters. javajava can belong to a regular expression of (java)*. A part of the inputted string that matches a capturing group will be saved and then recalled by back references.

Java provides numbering to identify capturing groups in a regular expression. They are numbered by counting their opening parentheses from left to right. For example, there are four following capturing groups in the regular expression ((A)(B(C))) :

- 1. ((A)(B(C)))
- 2. (A)
- 3. (B(C))
- 4. (C)

You can invoke the Matcher method groupCount() to determine how many capturing groups there are in a Matcher's Pattern.

The numbering of capturing groups is necessary to recall a stored part of a string by back references. A back reference is invoked by \n, where n is the index of a subgroup to recall the capturing group.

Table 3.3. Groups usage

Whole Content	Regular Expression	Result
abab	<code>([a-z][a-z])\1</code>	Found the text "abab" starting at index 0 and ending at index 4.
abcd	<code>([a-z][a-z])\1</code>	No match found.
abcd	<code>([a-z][a-z])</code>	Found the text "ab" starting at index 0 and ending at index 2. I found the text "cd" starting at index 2 and ending at index 4.

String.split()

J2SE 1.4 added the `split()` method to the `String` class to simplify the task of breaking a string into substrings, or tokens. This method uses the concept of a regular expression to specify the delimiters. A regular expression is a remnant from the Unix `grep` tool ("`grep`" meaning "general regular expression parser").

See most any introductory Unix text or the Java API documentation for the `java.util.regex.Pattern` class.

In its simplest form, searching for a regular expression consisting of a single character finds a match of that character. For example, the character 'x' is a match for the regular expression "x".

The `split()` method takes a parameter giving the regular expression to use as a delimiter and returns a `String` array containing the tokens so delimited. Using `split()` function:

```
String str = "This is a string object";
String[] words = str.split (" ");
for (String word : words) {
    out.println (word);
}
```

The output:

```
This
is
a
string
object
```

NOTE, `str.split (" ");` is equal to `str.split ("\\s");`.

To use "*" (which is a "special" regex character) as a delimiter, specify "*" as the regular expression (escape it):

```
String str = "A*bunch*of*stars";
String[] starwords = str.split ("\\*");
```

```
A
bunch
of
stars
```

NOTE, always use double "\\" for escaping in java source code, i.e. "\\s", "\\d", "*", otherwise the code will not compile:

```
String str = "boo and foo";
str.split("\s"); // WRONG ! Compilation error !
```

```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
    Invalid escape sequence (valid ones are \b \t \n \f \r \" \' \\ )

    at regex.Replacement.main(Replacement.java:15)
```

The following example (splitting by single character):

```
String str = "My1Daddy2cooks34pudding";
String[] words = str.split ("d");
for (String word : words) {
    System.out.println (word);
}
```

gives the following output:

```
My1Da
y2cooks34pu
ing
```

The same string, but with escaped "d" (regexp):

```
String str = "My1Daddy2cooks34pudding";
String[] words = str.split ("\\d"); // NOT "d"
for (String word : words) {
    System.out.println (word);
}
```

The output:

```
My
Daddy
cooks

pudding
```

```
public String[] split(String regex)
```

Splits this string around matches of the given regular expression. This method works as if by invoking the two-argument `split(...)` method with the given expression and a limit argument of zero. Trailing empty strings are therefore not included in the resulting array. The string "boo:and:foo", for example, yields the following results with these expressions:

```
String str = "boo:and:foo";
System.out.println(Arrays.toString(str.split(":")));
System.out.println(Arrays.toString(str.split("o")));
```

The output:

```
[boo, and, foo]
[b, , :and:f]
```

```
public String[] split(String regex, int limit)
```

Splits this string around matches of the given regular expression.

The array returned by this method contains each substring of this string that is terminated by another substring that matches the given expression or is terminated by the end of the string. The substrings in the array are in the order in which they occur in this string. If the expression does not match any part of the input then the resulting array has just one element, namely this string.

The `limit` parameter controls the number of times the pattern is applied and therefore affects the length of the resulting array. If the limit `n` is greater than zero then the pattern will be applied at most `n - 1` times, the array's length will be no greater than `n`, and the array's last entry will contain all input beyond the last matched delimiter. If `n` is non-positive then the pattern will be applied as many times as possible and the array can have any length. If `n` is zero then the pattern will be applied as many times as possible, the array can have any length, and trailing empty strings will be discarded:

```
String str = "boo:and:foo";
    System.out.println("a. " + Arrays.toString(str.split(":", 2)));
    System.out.println("b. " + Arrays.toString(str.split(":", 5)));
    System.out.println("c. " + Arrays.toString(str.split(":", -2)));
    System.out.println("d. " + Arrays.toString(str.split("o", 5)));
    System.out.println("e. " + Arrays.toString(str.split("o", -2)));
    System.out.println("f. " + Arrays.toString(str.split("o", 0)));
```

An invocation of this method of the form `str.split(regex, n)` yields the same result as the expression:

```
Pattern.compile(regex).split(str, n)
```

Formatted input

The scanner API provides basic input functionality for reading data from the system console or any data stream. The following example reads a `String` from standard input and expects a following `int` value:

```
Scanner s= new Scanner(System.in);
String param= s.next();
int value=s.nextInt();
s.close();
```

The Scanner methods like `next` and `nextInt` will block if no data is available. If you need to process more complex input, then there are also pattern-matching algorithms, available from the `java.util.Formatter` class.

`java.util.Scanner` is a simple text scanner which can parse primitive types and strings using regular expressions.

A Scanner breaks its input into tokens using a delimiter pattern, which by default matches whitespace. The resulting tokens may then be converted into values of different types using the various `next` methods.

For example, this code allows a user to read a number from `System.in`:

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
```

As another example, this code allows long types to be assigned from entries in a file myNumbers:

```
Scanner sc = new Scanner(new File("myNumbers"));
while (sc.hasNextLong()) {
    long aLong = sc.nextLong();
}
```

The scanner can also use delimiters other than whitespace. This example reads several items from a string:

```
String input = "1 fish 2 fish red fish blue fish";
Scanner s = new Scanner(input).useDelimiter("\\s*fish\\s*");
System.out.println(s.nextInt());
System.out.println(s.nextInt());
System.out.println(s.next());
System.out.println(s.next());
s.close();
```

prints the following output:

```
1
2
red
blue
```

The same output can be generated with this code, which uses a regular expression to parse all four tokens at once:

```
String input = "1 fish 2 fish red fish blue fish";
Scanner s = new Scanner(input);
s.findInLine("(\\d+) fish (\\d+) fish (\\w+) fish (\\w+)");
MatchResult result = s.match();
for (int i=1; i<=result.groupCount(); i++) {
    System.out.println(result.group(i));
}
s.close();
```

Class `java.util.Scanner` implements a simple text scanner (lexical analyzer) which uses regular expressions to parse primitive types and strings from its source.

A Scanner converts the input from its source into tokens using a delimiter pattern, which by default matches whitespace.

The tokens can be converted into values of different types using the various `next()` methods:

```
Scanner scanner = new Scanner(System.in); // Connected to standard input.
int i = scanner.nextInt();
```

```
Scanner scanner = new Scanner(new File("myLongNumbers")); (1) Construct a scanner.
while (scanner.hasNextLong()) { // (2) End of input? May block.
```

```
        long aLong = scanner.nextLong(); // (3) Deal with the current token. May block.
    }
    scanner.close(); // (4) Closes the scanner. May close the source.
```

Before parsing the next token with a particular `next()` method, for example at (3), a lookahead can be performed by the corresponding `hasNext()` method as shown at (2).

The `next()` and `hasNext()` methods and their primitive-type companion methods (such as `nextInt()` and `hasNextInt()`) first skip any input that matches the delimiter pattern, and then attempt to return the next token.

Constructing a **Scanner**

A scanner must be constructed to parse text:

```
Scanner(Type source)
```

Returns an appropriate scanner. `Type` can be a `String`, a `File`, an `InputStream`, a `ReadableByteChannel`, or a `Readable` (implemented by `CharBuffer` and various `Readers`).

Scanning

A scanner throws an `InputMismatchException` when the next token cannot be translated into a valid value of requested type.

Lookahead methods:

```
// returns true if this scanner has another token in its input
boolean hasNext()

// returns true if the next token matches the specified pattern
boolean hasNext(Pattern pattern)

// returns true if the next token matches the pattern constructed
// from the specified string
boolean hasNext(String pattern)

// returns true if the next token in this scanner's input can be interpreted as an
// numeric type value corresponding to 'XXX' in the default or specified
// radix
boolean hasNextXXX()
boolean hasNextXXX(int radix)

// returns true if the next token in this scanner's
// input can be interpreted as a boolean value using
// a case insensitive pattern created from the string
// "true|false"
boolean hasNextBoolean()
```

The name `XXX` can be: `Byte`, `Short`, `Int`, `Long`, `Float`, `Double` or `BigInteger`.

Parsing the next token methods:

```
// scans and returns the next complete token from this scanner
String next()

// returns the next string in the input that matches the specified pattern
String next(Pattern pattern)
```

```
// returns the next token if it matches the pattern constructed from the specified string
String next(String pattern)

// scans the next token of the input as a 'xxx' value corresponding to 'XXX'
xxx nextXXX()
xxx nextXXX(int radix)

// scans the next token of the input into a boolean
// value and returns that value
boolean nextBoolean()

// advances this scanner past the current line and
// returns the input that was skipped
String nextLine()
```

The name XXX can be: Byte, Short, Int, Long, Float, Double or BigInteger. The corresponding 'xxx' can be: byte, short, int, long, float, double or BigInteger.

Example:

```
String input = "123 45,56 TRUE 567 722 blabla";
Scanner scanner = new Scanner(input);
out.println(scanner.hasNextInt());
out.println(scanner.nextInt());
out.println(scanner.hasNextDouble());
out.println(scanner.nextDouble());
out.println(scanner.hasNextBoolean());
out.println(scanner.nextBoolean());
out.println(scanner.hasNextInt());
out.println(scanner.nextInt());
out.println(scanner.hasNextLong());
out.println(scanner.nextLong());
out.println(scanner.hasNext());
out.println(scanner.next());
out.println(scanner.hasNext());
scanner.close();
```

The output:

```
true
123
true
45.56
true
true
true
567
true
722
true
blabla
false
```

Error in parsing:

```
String input = "123,123";
Scanner scanner = new Scanner(input);
```

```
out.println(scanner.hasNextInt());
out.println(scanner.nextInt());
scanner.close();
```

The output (runtime exception):

```
false
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Unknown Source)
    at java.util.Scanner.next(Unknown Source)
    ...
```

Formatted output

Developers now have the option of using printf-type functionality to generate formatted output. This will help migrate legacy C applications, as the same text layout can be preserved with little or no change.

Most of the common C printf formatters are available, and in addition some Java classes like Date and BigInteger also have formatting rules. See the `java.util.Formatter` class for more information. Although the standard UNIX newline '\n' character is accepted, for cross-platform support of newlines the Java %n is recommended. Furthermore, J2SE 5.0 added a `printf()` method to the `PrintStream` class. So now you can use `System.out.printf()` to send formatted numerical output to the console. It uses a `java.util.Formatter` object internally:

```
System.out.printf("name count%n");
System.out.printf("%s %5d%n", user, total);
```

The simplest of the overloaded versions of the method goes as

```
printf (String format, Object... args)
```

The format argument is a string in which you embed specifier substrings that indicate how the arguments appear in the output. For example:

```
double pi = Math.PI;
System.out.printf ("1. pi = %5.3f %n", pi);
System.out.printf ("2. pi = %f      %n", pi);
System.out.printf ("3. pi = %b      %n", pi);
System.out.printf ("4. pi = %s      %n", pi);
```

results in the console output:

```
1. pi = 3,142
2. pi = 3,141593
3. pi = true
4. pi = 3.141592653589793
```

The format string includes the specifier "%5.3f" that is applied to the argument. The '%' sign signals a specifier. The width value 5 requires at least five characters for the number, the precision value 3 requires three places in the fraction, and the conversion symbol 'f' indicates a decimal representation of a floating-point number.

A specifier needs at least the conversion character, of which there are several besides 'f'. Some of the other conversions

include:

- %b - If the argument `arg` is `null`, then the result is "false". If `arg` is a boolean or `Boolean`, then the result is the string returned by `String.valueOf()`. Otherwise, the result is "true".
- %c - The result is a Unicode character.
- %d - The result is formatted as a decimal integer.
- %f - The result is formatted as a decimal number.
- %s - If the argument `arg` is `null`, then the result is "null". If `arg` implements `Formattable`, then `arg.formatTo` is invoked. Otherwise, the result is obtained by invoking `arg.toString()`.

There are also special conversions for dates and times. The general form of the specifier includes several optional terms:

```
%[argument_index$][flags][width][.precision]conversion
```

The `argument_index` indicates to which argument the specifier applies. For example, `%2$` indicates the second argument in the list. A flag indicates an option for the format. For example, `'+'` requires that a sign be included and `'0'` requires padding with zeros. The width indicates the minimum number of characters and the precision is the number of places for the fraction.

There is also one specifier that doesn't correspond to an argument. It is `"%n"` which outputs a line break. A `"\n"` can also be used in some cases, but since `"%n"` always outputs the correct platform-specific line separator, it is portable across platforms whereas `"\n"` is not.

Classes `java.lang.String`, `java.io.PrintStream`, `java.io.PrintWriter` and `java.util.Formatter` provide the following overloaded methods for formatted output:

```
// Writes a formatted string using the specified format string and argument list
format(String format, Object... args)
format(Locale l, String format, Object... args)
```

The `format()` method returns a `String`, a `PrintStream`, a `PrintWriter` or a `Formatter` respectively for these classes, allowing method call chaining.

The `format()` method is static in the `String` class.

In addition, classes `PrintStream` and `PrintWriter` provide the following convenience methods:

```
// Writes a formatted string using the specified format string and argument list
printf(String format, Object... args)
printf(Locale l, String format, Object... args)
```

Format string syntax provides support for layout justification and alignment, common formats for numeric, string, and date/time data, and locale-specific output.

Class `java.util.Formatter` provides the core support for formatted output.

Format string syntax

The format string can specify fixed text and embedded format specifiers:

```
out.printf("Formatted output|%6d|%8.3f|%.2f%n", 2005, Math.PI, 1234.0354);
```

The output will be (default locale Russian):

```
Formatted output| 2005| 3,142|1234,04
```

The format string is the first argument.

It contains three format specifiers `%6d`, `%8.3f`, and `%.2f` which indicate how the arguments should be processed and where the arguments should be inserted in the format string.

All other text in the format string is fixed, including any other spaces or punctuation.

The argument list consists of all arguments passed to the method after the format string. In the above example, the argument list is of size three.

In the above example, the first argument is formatted according to the first format specifier, the second argument is formatted according to the second format specifier, and so on.

Format specifiers for general, character, and numeric types

```
%[argument_index$][flags][width][.precision]conversion
```

The characters `%`, `$` and `.` have special meaning in the context of the format specifier.

The optional `argument_index` is a decimal integer indicating the position of the argument in the argument list. The first argument is referenced by `"1$"`, the second by `"2$"`, and so on.

The optional `flags` is a set of characters that modify the output format. The set of valid flags depends on the conversion.

The optional `width` is a decimal integer indicating the minimum number of characters to be written to the output.

The optional `precision` is a decimal integer usually used to restrict the number of characters. The specific behavior depends on the conversion.

The required `conversion` is a character indicating how the argument should be formatted. The set of valid conversions for a given argument depends on the argument's data type.

Conversion categories (required for exam only)

General ('b', 's'): May be applied to any argument type.

Character ('c'): May be applied to basic types which represent unicode characters: `char`, `Character`, `byte`, `Byte`, `short`, and `Short`.

Numeric integral ('d'): May be applied to integral types: `byte`, `Byte`, `short`, `Short`, `int`, `Integer`, `long`, `Long`, and `BigInteger`.

```
out.printf("%8d", 10.10); // WRONG !!!
```

```
java.util.IllegalFormatConversionException: d != java.lang.Double
```

```
out.printf("%8d", (int) 10.10); // OK !
```

Numeric floating point ('f'): May be applied to floating-point types: float, Float, double, Double, and BigDecimal.

```
out.printf("%8.3f", 10); // WRONG !!!
```

```
java.util.IllegalFormatConversionException: f != java.lang.Integer
```

Conversion rules

'b' - If the argument arg is null, then the result is "false". If arg is a boolean or Boolean, then the result is string returned by String.valueOf(). Otherwise, the result is "true".

'c' - The result is a Unicode character.

'd' - The result is formatted as a decimal integer.

'f' - The result is formatted as a floating point decimal number.

's' - If the argument arg is null, then the result is "null". If arg implements Formattable, then arg.formatTo() is invoked. Otherwise, the result is obtained by invoking arg.toString().

Precision

For general argument types, the precision is the maximum number of characters to be written to the output:

```
out.printf("|%8.3s|", "2005");
```

```
|    200|
```

For the floating-point conversions: if the conversion is 'f', then the precision is the number of digits after the decimal separator:

```
out.printf("|%8.3f| %n", 2005.1234);  
out.printf("|%8.3s|", true);
```

```
|2005,123|  
|    tru|
```

For character ('c') and integral ('d') argument types the precision is not applicable. If a precision is provided, an exception will be thrown:

```
out.printf("|%8.3d|", 2005);
```

Exception in thread "main" java.util.IllegalFormatPrecisionException: 3

[Prev](#)

[Up](#)

[Next](#)

Use standard J2SE APIs in the java.text package to correctly format or parse dates, numbers, and currency values for a specific locale; and, given a scenario, determine the appropriate methods to use if you want to use the default locale or a specific locale. Describe the purpose and use of the java.util.Locale class.

Chapter 4. Concurrency

[Home](#)



SCDJWS 1.4
Quiz



ICAD
Study Guide



Magnet
Mocker

SCWCD 1.4
Study Guide

SCBCD 1.3
Study Guide

SCDJWS 1.4
Study Guide

IBM Test 287
Study Guide